

Dataset Link: <https://www.kaggle.com/datasets/sachinpatel21/az-handwritten-alphabets-in-csv-format>

A_Z Handwritten Data.csv

225,255,18,0,0,0,0,0,0,0,0,0,0,0,0,0,0,144,223,130,0,0,0,0,0,0,170,255,101
0,0,0,0,0,0,23,227,255,255,255,210,0,0,0,0,31,255,255,121,0,0,0,0,0,0,0,0,0
239,149,0,0,0,0,0,0,0,0,0,0,90,223,255,127,24,0,0,0,0,0,0,0,61,195,253,190
0,0,0,127,164,56,1,0,0,0,0,13,121,210,89,0,0,0,0,0,0,0,0,0,0,0,0,0,31,193
129,0,0,0,0,0,0,0,0,0,0,39,82,0,0,0,0,0,0,0,0,0,0,140,248,41,0,0,0,0,0,20
0,174,255,228,13,0,0,0,0,0,0,0,0,49,117,191,229,255,255,255,154,33,0,0,0
151,12,0,0,0,0,0,0,0,0,0,0,0,0,0,0,189,255,218,84,6,0,0,0,64,179,151,12,0
255,210,22,0,0,0,0,0,0,0,0,0,0,32,219,255,182,0,0,0,0,0,0,0,0,9,203,20,0,0
0,0,20,255,0,0,0,0,0,0,0,0,0,0,65,254,131,226,156,0,0,0,0,0,0,0,0,65,207
255,251,85,0,0,0,0,0,0,0,0,0,0,0,255,255,255,156,0,0,0,0,0,0,30,231,255
0,0,0,0,0,0,0,122,228,223,77,0,0,0,0,0,11,104,243,211,80,0,0,0,0,0,0,0,0
120,1,0,0,0,0,0,0,0,0,0,0,0,2,95,217,228,92,8,0,0,0,0,0,0,69,221,199,30,0,0
174,255,223,4,0,0,0,0,0,0,0,5,234,188,0,0,0,0,0,0,0,0,0,0,0,0,0,170,90,0,0,0
208,0,0,0,0,0,0,0,0,0,1,53,215,255,243,157,7,0,103,217,97,255,216,73,47,60
0,0,5,180,249,240,78,0,0,0,0,0,0,15,115,249,193,33,0,0,0,0,0,0,0,0,0,0,0,50,198
252,219,109,20,0,0,0,0,62,206,200,32,0,0,0,0,0,0,0,0,0,0,0,19,166,245,228,99
255,255,0,0,0,0,0,0,0,0,0,0,146,255,255,219,0,0,0,0,0,0,0,0,164,255,255,200
0,0,0,0,60,210,253,249,177,53,0,0,0,0,1,71,165,188,255,219,90,0,0,0,0,0,0,0,0
177,255,255,185,0,0,0,0,0,0,0,0,0,0,100,253,255,239,114,0,0,0,0,0,0,0,177,55
174,0,0,0,0,0,0,0,0,223,255,255,234,14,0,0,0,0,0,0,109,255,255,174,82,3,32,0
149,255,255,95,0,0,0,0,0,0,32,229,255,238,47,0,0,0,0,0,0,0,0,0,0,160,255,255
0,0,0,0,0,0,0,0,230,255,19,0,0,0,0,0,0,138,255,186,0,0,0,0,0,0,0,0,0,0,799
142,0,0,0,0,0,0,54,253,255,255,90,0,0,0,0,0,0,0,0,0,0,72,249,255,84,0,46,70,8
0,0,0,0,0,0,0,0,0,0,0,0,11,170,246,164,44,0,0,0,0,10,102,229,199,69,0,0,0,0,0,0,
45,166,242,161,16,0,0,0,0,0,0,0,0,0,60,215,205,56,0,0,0,0,0,0,57,196,234,135
255,203,3,0,0,0,0,0,25,249,255,255,0,0,0,0,0,0,0,0,0,0,0,0,17,255,255,84,0,0,0
43,0,0,0,0,0,0,0,0,0,0,0,0,0,61,33,0,0,0,0,0,0,0,0,33,233,118,0,0,0,0,0,0,0,0

PROGRAM

```
import cv2

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.utils import shuffle

from keras.models import Sequential

from keras.layers import Dense, Flatten, Conv2D, MaxPool2D, Dropout

from keras.optimizers import Adam

from keras.callbacks import ReduceLROnPlateau, EarlyStopping

from keras.utils import to_categorical


data = pd.read_csv(r"A_Z Handwritten Data.csv").astype('float32')
X = data.drop('0', axis=1)
y = data['0']

train_x, test_x, train_y, test_y = train_test_split(X, y, test_size=0.2)

train_x = np.reshape(train_x.values, (train_x.shape[0], 28, 28))
test_x = np.reshape(test_x.values, (test_x.shape[0], 28, 28))

word_dict = {i: chr(65 + i) for i in range(26)}

y_int = np.int8(y)

count = np.zeros(26, dtype='int')

for i in y_int:
    count[i] += 1

alphabets = [word_dict[i] for i in range(26)]

fig, ax = plt.subplots(1, 1, figsize=(10, 10))

ax.barh(alphabets, count)

plt.xlabel("Number of Elements")

plt.ylabel("Alphabets")

plt.title("Distribution of Characters")
```

```

plt.minorticks_on()
plt.grid(which='major', linestyle='-', linewidth='0.5', color='red')
plt.grid(which='minor', linestyle=':', linewidth='0.5', color='black')
plt.show()
shuff = shuffle(train_x[:100])
fig, ax = plt.subplots(3, 3, figsize=(10, 10))
axes = ax.flatten()
for i in range(9):
    _, img_bin = cv2.threshold(shuff[i], 30, 200, cv2.THRESH_BINARY)
    axes[i].imshow(img_bin, cmap=plt.get_cmap('gray'))
plt.show()

```

```

train_X = train_x.reshape(train_x.shape[0], 28, 28, 1)
test_X = test_x.reshape(test_x.shape[0], 28, 28, 1)
print("Train data shape:", train_X.shape)
print("Test data shape:", test_X.shape)
train_yOHE = to_categorical(train_y, num_classes=26)
test_yOHE = to_categorical(test_y, num_classes=26)
print("Train labels shape:", train_yOHE.shape)
print("Test labels shape:", test_yOHE.shape)

```

```

model = Sequential([
    Conv2D(filters=32, kernel_size=(3, 3), activation='relu', input_shape=(28, 28, 1)),
    MaxPool2D(pool_size=(2, 2), strides=2),
    Conv2D(filters=64, kernel_size=(3, 3), activation='relu', padding='same'),
    MaxPool2D(pool_size=(2, 2), strides=2),
    Conv2D(filters=128, kernel_size=(3, 3), activation='relu', padding='valid'),
    MaxPool2D(pool_size=(2, 2), strides=2),
    Flatten(),
    Dense(64, activation="relu"),

```

```

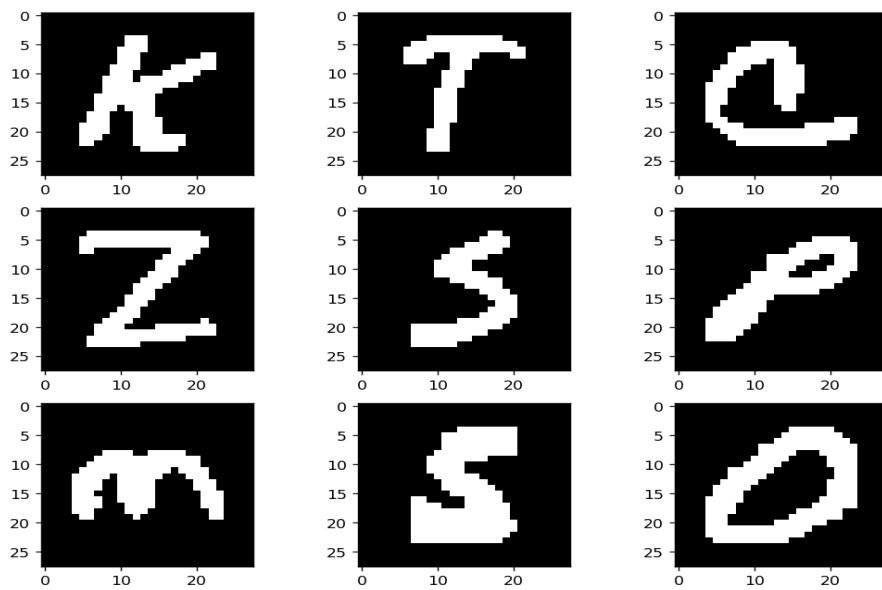
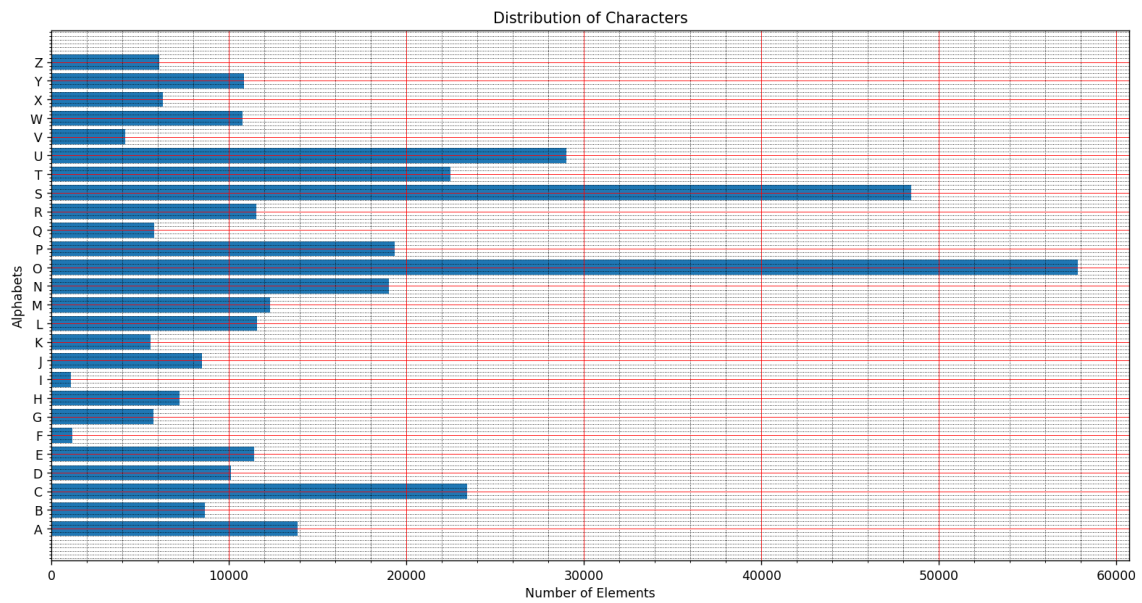
Dense(128, activation="relu"),
Dense(26, activation="softmax")
])
model.compile(optimizer=Adam(learning_rate=0.001), loss='categorical_crossentropy',
metrics=['accuracy'])
history = model.fit(train_X, train_yOHE, epochs=15, validation_data=(test_X, test_yOHE))
model.summary()
model.save(r'model_hand.h5')
print("Validation Accuracy:", history.history['val_accuracy'])
print("Training Accuracy:", history.history['accuracy'])
print("Validation Loss:", history.history['val_loss'])
print("Training Loss:", history.history['loss'])
fig, axes = plt.subplots(3, 3, figsize=(8, 9))
axes = axes.flatten()
for i, ax in enumerate(axes):
    img = np.reshape(test_X[i], (28, 28))
    ax.imshow(img, cmap=plt.get_cmap('gray'))
    pred = word_dict[np.argmax(test_yOHE[i])]
    ax.set_title("Prediction: " + pred)
plt.show()
img = cv2.imread(r'test_image.png')
if img is None:
    print("Error: Image not found. Check file path.")
    exit()

img_copy = img.copy()
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
img = cv2.resize(img, (400, 440))
img_copy = cv2.GaussianBlur(img_copy, (7, 7), 0)
img_gray = cv2.cvtColor(img_copy, cv2.COLOR_BGR2GRAY)

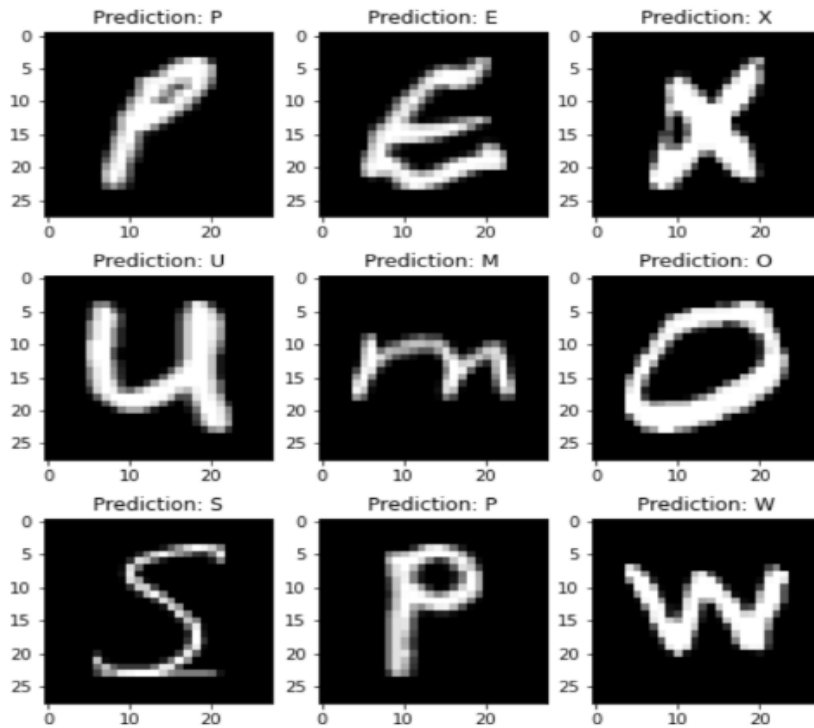
```

```
_, img_thresh = cv2.threshold(img_gray, 100, 255, cv2.THRESH_BINARY_INV)
img_final = cv2.resize(img_thresh, (28, 28))
img_final = np.reshape(img_final, (1, 28, 28, 1))
img_pred = word_dict[np.argmax(model.predict(img_final))]
cv2.putText(img, "Image Data", (100, 25), cv2.FONT_HERSHEY_DUPLEX, fontScale=1,
thickness=2, color=(255, 0, 0))
cv2.putText(img, "Character Prediction: " + img_pred, (10, 410),
cv2.FONT_HERSHEY_SIMPLEX,
fontScale=1, thickness=2, color=(0, 0, 255))
cv2.imshow('Character Recognition', img)
while True:
    k = cv2.waitKey(1) & 0xFF
    if k == 27:
        break
cv2.destroyAllWindows()
```

OUTPUT



```
Train data shape: (297960, 28, 28, 1)
Test data shape: (74490, 28, 28, 1)
Train labels shape: (297960, 26)
Test labels shape: (74490, 26)
```



9312/9312 494s 52ms/step - accuracy: 0.9580 - loss: 0.1562 - val_accuracy: 0.9764 - val_loss: 0.0854
Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 13, 13, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 64)	0
conv2d_2 (Conv2D)	(None, 4, 4, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 2, 2, 128)	0
flatten (Flatten)	(None, 512)	0
dense (Dense)	(None, 64)	32,832
dense_1 (Dense)	(None, 128)	8,320
dense_2 (Dense)	(None, 26)	3,354

Total params: 411,536 (1.57 MB)
Trainable params: 137,178 (535.85 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 274,358 (1.05 MB)

Validation Accuracy: [0.9768694043159485, 0.9812458157539368, 0.9830044507980347]
Training Accuracy: [0.9566921591758728, 0.979554295539856, 0.9831856489181519]
Validation Loss: [0.08225011825561523, 0.07045286893844604, 0.06703901290893555]
Training Loss: [0.15571142733097076, 0.07376661151647568, 0.06095144897699356]
1/1 0s 149ms/step

PROGRAM

```
import numpy as np
import pandas as pd
from sklearn.datasets import fetch_lfw_people
faces = fetch_lfw_people(
    min_faces_per_person=100,
    resize=1.0,
    slice_=(slice(60, 188), slice(60, 188)),
    color=True)
class_count = len(faces.target_names)
print(faces.target_names)
print(faces.images.shape)
import matplotlib.pyplot as plt
import seaborn as sns
sns.set()
fig, ax = plt.subplots(3, 6, figsize=(18, 10))
for i, axi in enumerate(ax.flat):
    axi.imshow(faces.images[i]) # Scale pixel values so Matplotlib doesn't clip everything
    above 1.0
    axi.set(
        xticks=[],
        yticks=[],
        xlabel=faces.target_names[faces.target[i]])
plt.show()
from collections import Counter
counts = Counter(faces.target)
names = {}
for key in counts.keys():
    names[faces.target_names[key]] = counts[key]
df = pd.DataFrame.from_dict(names, orient='index')
df.plot(kind='bar')
```



```

mask = np.zeros(faces.target.shape, dtype=np.bool)
for target in np.unique(faces.target):
    mask[np.where(faces.target == target)[0][:100]] = 1
x_faces = faces.data[mask]
y_faces = faces.target[mask]
x_faces = np.reshape(x_faces,(
    x_faces.shape[0],
    faces.images.shape[1],
    faces.images.shape[2],
    faces.images.shape[3]))
x_faces.shape
from tensorflow.keras.utils import to_categorical
from sklearn.model_selection import train_test_split
face_images = x_faces / 255 # Normalize pixel values
face_labels = to_categorical(y_faces)
x_train, x_test, y_train, y_test = train_test_split(
    face_images,
    face_labels,
    train_size=0.8,
    stratify=face_labels,
    random_state=0)
from keras.layers import Dense
from keras.models import Sequential
from keras.layers import Conv2D
from keras.layers import MaxPooling2D
from keras.layers import Flatten
model = Sequential()
model.add(Conv2D(32,(3, 3),activation='relu',input_shape=(face_images.shape[1:])))
model.add(MaxPooling2D(2, 2))
model.add(

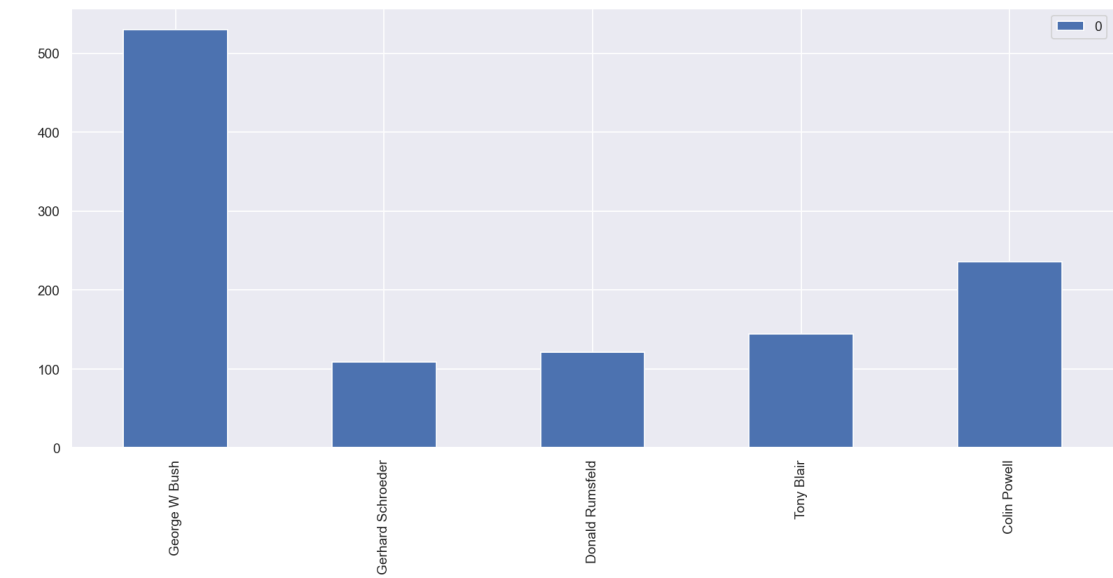
```

```
Conv2D(
    64,
    (3, 3),
    activation='relu'))
model.add(MaxPooling2D(2, 2))
model.add(Conv2D(64,(3, 3),activation='relu'))
model.add(MaxPooling2D(2, 2))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dense(class_count, activation='softmax'))
model.compile(optimizer='adam',loss='categorical_crossentropy',metrics=['accuracy'])
model.summary()
hist = model.fit(
    x_train,
    y_train,
    validation_data=(x_test, y_test),
    epochs=20,
    batch_size=25)
acc = hist.history['accuracy']
val_acc = hist.history['val_accuracy']
epochs = range(1, len(acc) + 1)
plt.plot(epochs, acc, '-', label='Training Accuracy')
plt.plot(epochs, val_acc, ':', label='Validation Accuracy')
plt.title('Training and Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend(loc='lower right')
plt.plot()
plt.show()
from sklearn.metrics import confusion_matrix
```

```
y_predicted = model.predict(x_test)
mat = confusion_matrix(
    y_test.argmax(axis=1),
    y_predicted.argmax(axis=1))
sns.heatmap(
    mat.T,
    square=True,
    annot=True,
    fmt='d',
    cbar=False,
    cmap='Blues',
    xticklabels=faces.target_names,
    yticklabels=faces.target_names)
plt.xlabel('Predicted label')
plt.ylabel('Actual label')
plt.show()
import keras.utils as image
x = image.load_img(
    'george.jpg',
    target_size=(face_images.shape[1:]))
plt.xticks([])
plt.yticks([])
plt.imshow(x)
plt.show()
x = image.img_to_array(x)/255
x = np.expand_dims(x, axis=0)
y = model.predict(x)[0]
for i in range(len(y)):
    print(faces.target_names[i] + ':' + str(y[i]))
```

OUTPUT

```
['Colin Powell' 'Donald Rumsfeld' 'George W Bush' 'Gerhard Schroeder'
'Tony Blair']
(1140, 128, 128, 3)
```



Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_1 (Conv2D)	(None, 61, 61, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_2 (Conv2D)	(None, 28, 28, 64)	36,928
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 64)	0
flatten (Flatten)	(None, 12544)	0
dense (Dense)	(None, 128)	1,605,760
dense_1 (Dense)	(None, 5)	645

Total params: 1,662,725 (6.34 MB)

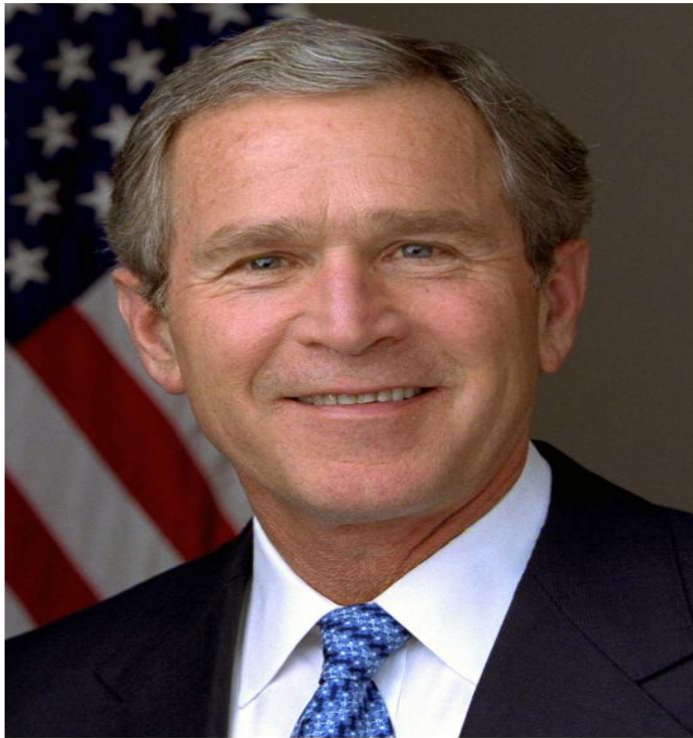
Trainable params: 1,662,725 (6.34 MB)

Non-trainable params: 0 (0.00 B)

```
Epoch 1/20
16/16 ━━━━━━━━━━━ 8s 371ms/step - accuracy: 0.2000 - loss: 1.6119 - val_accuracy: 0.2000 - val_loss: 1.6094
Epoch 2/20
16/16 ━━━━━━━━━━━ 5s 313ms/step - accuracy: 0.1700 - loss: 1.6097 - val_accuracy: 0.2000 - val_loss: 1.6094
Epoch 3/20
16/16 ━━━━━━━━━━━ 5s 305ms/step - accuracy: 0.2000 - loss: 1.6096 - val_accuracy: 0.2000 - val_loss: 1.6094
Epoch 4/20
16/16 ━━━━━━━━━━━ 6s 386ms/step - accuracy: 0.2000 - loss: 1.6095 - val_accuracy: 0.2000 - val_loss: 1.6094
Epoch 5/20
16/16 ━━━━━━━━━━━ 8s 522ms/step - accuracy: 0.1750 - loss: 1.6097 - val_accuracy: 0.2000 - val_loss: 1.6094
Epoch 6/20
16/16 ━━━━━━━━━━━ 8s 334ms/step - accuracy: 0.1950 - loss: 1.6097 - val_accuracy: 0.2000 - val_loss: 1.6094
Epoch 7/20
16/16 ━━━━━━━━━━━ 7s 416ms/step - accuracy: 0.1850 - loss: 1.6096 - val_accuracy: 0.2000 - val_loss: 1.6094
Epoch 8/20
16/16 ━━━━━━━━━━━ 11s 711ms/step - accuracy: 0.1725 - loss: 1.6096 - val_accuracy: 0.2000 - val_loss: 1.6094
Epoch 9/20
16/16 ━━━━━━━━━━━ 18s 529ms/step - accuracy: 0.1650 - loss: 1.6096 - val_accuracy: 0.2000 - val_loss: 1.6094
Epoch 10/20
16/16 ━━━━━━━━━━━ 9s 588ms/step - accuracy: 0.1850 - loss: 1.6096 - val_accuracy: 0.2000 - val_loss: 1.6094
Epoch 11/20
16/16 ━━━━━━━━━━━ 9s 571ms/step - accuracy: 0.1850 - loss: 1.6096 - val_accuracy: 0.2000 - val_loss: 1.6094
Epoch 12/20
16/16 ━━━━━━━━━━━ 10s 579ms/step - accuracy: 0.1775 - loss: 1.6095 - val_accuracy: 0.2000 - val_loss: 1.6094
Epoch 13/20
16/16 ━━━━━━━━━━━ 13s 775ms/step - accuracy: 0.2000 - loss: 1.6095 - val_accuracy: 0.2000 - val_loss: 1.6094
Epoch 14/20
16/16 ━━━━━━━━━━━ 14s 864ms/step - accuracy: 0.2000 - loss: 1.6095 - val_accuracy: 0.2000 - val_loss: 1.6094
Epoch 15/20
16/16 ━━━━━━━━━━━ 12s 727ms/step - accuracy: 0.1750 - loss: 1.6095 - val_accuracy: 0.2000 - val_loss: 1.6094
Epoch 16/20
16/16 ━━━━━━━━━━━ 12s 759ms/step - accuracy: 0.1750 - loss: 1.6095 - val_accuracy: 0.2000 - val_loss: 1.6094
Epoch 17/20
16/16 ━━━━━━━━━━━ 19s 624ms/step - accuracy: 0.1925 - loss: 1.6095 - val_accuracy: 0.2000 - val_loss: 1.6094
Epoch 18/20
16/16 ━━━━━━━━━━━ 7s 425ms/step - accuracy: 0.1750 - loss: 1.6096 - val_accuracy: 0.2000 - val_loss: 1.6094
Epoch 19/20
16/16 ━━━━━━━━━━━ 6s 407ms/step - accuracy: 0.2000 - loss: 1.6095 - val_accuracy: 0.2000 - val_loss: 1.6094
Epoch 20/20
16/16 ━━━━━━━━━━━ 6s 396ms/step - accuracy: 0.1725 - loss: 1.6096 - val_accuracy: 0.2000 - val_loss: 1.6094
```



Actual Label	Colin Powell	0	20	0	0	0
	Donald Rumsfeld	0	20	0	0	0
	George W Bush	0	20	0	0	0
	Gerhard Schroeder	0	20	0	0	0
	Tony Blair	0	20	0	0	0
		Colin Powell	Donald Rumsfeld	George W Bush	Gerhard Schroeder	Tony Blair
		Predicted Label				



--- Prediction Probabilities ---

Colin Powell: 0.1993

Donald Rumsfeld: 0.1878

George W Bush: 0.1732

Gerhard Schroeder: 0.2614

Tony Blair: 0.1783